OPEN CLUSTER DESIGNS ALIGNED AI INFERENCE FABRIC REFERENCE ARCHITECTURE

Author (s) :

Art Fewell, Hedgehog Inc.

Arun Solleti, Celestica Inc.

Date: April 2026

Version History

Date	Version #	Author	Description
27 MAR 2026	1.0	Art Fewell & Arun Solleti	Initial Draft
10 APR 2026	1.1	Art Fewell & Arun Solleti	Updated with review comments
26 APR 2026	1.2	Art Fewell & Arun Solleti	Updated with Steering Committee Feedback

Date: April 2026

Executive Summary

AI inference is rapidly becoming the dominant operational workload in accelerator clusters, yet inference fabrics face a distinct set of challenges: multi-tenant isolation with per-tenant gateway services, north-south traffic patterns that differ fundamentally from training's east-west collectives, and a wide range of deployment profiles — from single-server model serving with no back-end network to distributed inference requiring lossless RDMA across hosts. The Open Compute Project (OCP) Open Cluster Designs System Architectures [1][2] define the structural building blocks but leave the network-centric design — switch management, tenant overlay policy, gateway integration, and lifecycle automation — to implementers. This document fills that gap with an inference-optimized fabric architecture built on open, SONiC-based switches managed by a Kubernetes-native control plane.

This Reference Architecture defines an inference-optimized Ethernet fabric for multi-tenant AI serving, aligned with the OCP Open Pod Group (OPG-M) and xPU Open Cluster (XOC-N) System Architectures [1][2], targeting 64–1,024 xPU clusters. The fabric is managed by Hedgehog, a Kubernetes-native control plane that treats switches as data-plane elements continuously reconciled from declarative Custom Resource Definitions (CRDs) — a GitOps-compatible model in which wiring, tenant overlays, and external connectivity are all expressed as versioned, reviewable intent. SONiC runs on OCP-compliant switches (Celestica DS5000 as the reference platform), with lifecycle automation spanning Day-0 Zero-Touch Provisioning, Day-1 topology and overlay onboarding, and Day-2 continuous reconciliation and observability.

Two profiles address the range of inference workloads. Minimal Inference provides a converged scale-out and storage fabric with no back-end network — right-sized for single-server model serving where all traffic flows north-south through the gateway. Distributed Inference adds a lossless RDMA back-end for cross-host parallelism, enabled via a single fabric-level toggle, while retaining the same converged frontend. Both profiles include a distributed x86 gateway for north-south traffic with stateful ingress filtering and horizontal scaling, and multi-tenant isolation

Date: April 2026

through per-tenant VRFs with VXLAN-backed subnets — shareable where policy permits, isolated by default. Each composition ships with connectivity maps, bills of materials, and Hedgehog Kubernetes CRDs as ready-to-deploy artifacts, and a companion VLAB environment lets operators validate topology and overlay intent on virtual SONiC switches before committing to physical infrastructure.

The underlying Hedgehog platform and SONiC-based fabric design have been validated in production deployments, including FarmGPU's XOC-512 cluster of NVIDIA B200 nodes — independently validated by SemiAnalysis, which confirmed the fabric outperformed several alternative network solutions [10]. The inference-specific composition variants in this RA extend that proven foundation to the distinct topology, gateway, and multi-tenancy requirements of AI serving workloads, providing operators with a complete, reproducible, and open starting point for inference fabric deployments at OPG and XOC scale.

Table of Contents

Date: April 2026

Date: April 2026

Date: April 2026

1. Introduction

AI inference workloads place distinct demands on the network. Where training fabrics optimize for massive east-west bandwidth across thousands of xPUs, inference clusters emphasize north-south throughput, multi-tenant isolation, gateway services for ingress/egress, and operational simplicity -- at the 64--1,024 xPU scale. This Reference Architecture addresses those demands using Hedgehog AI Network as the Kubernetes-native control plane and SONiC as the network operating system on OCP-compliant switches.

Hedgehog manages the entire fabric lifecycle through declarative Kubernetes CRDs. Operators define physical topology using the Wiring API (Switch, Server, Connection CRDs) and express tenant overlays, external connectivity, and gateway services using the VPC API (VPC, VPCAttachment, VPCPeering, External, ExternalAttachment, ExternalPeering CRDs). The Hedgehog controller continuously reconciles this declared intent to SONiC switches via per-switch agents, treating every switch as a data-plane element -- no manual CLI, no device-level templates. The underlying platform and fabric design have been validated in production, including FarmGPU (validated by SemiAnalysis) [10]; the composition variants defined here build on that proven foundation.

Two OCP companion specifications provide the structural foundation. The OPG-M System Architecture [1] defines the Open Pod Group -- a modular building block of M xPUs (for example, OPG-64: 8 eight-xPU servers with networking, storage, and management). The XOC-N System Architecture [2] defines the xPU Open Cluster, which composes OPG pods under shared spine and aggregation layers into a scale-out cluster. This Reference Architecture provides the inference-optimized, network-centric complement to those specifications.

Two profiles span common inference scenarios. *Minimal Inference* provides a converged SO-C and storage fabric without a back-end network, right-sized for single-server model serving. **Distributed Inference** adds a scale-out back-end for cross-host parallelism (model sharding, distributed batching, shared caches), with lossless RDMA transport enabled by a single switch-

level toggle, while retaining the same converged SO-C and storage fabric. Both profiles include the distributed Hedgehog Gateway for NAT/PAT services, VPC-based multi-tenant isolation with EVPN/VXLAN overlays, and full lifecycle automation from ZTP through continuous reconciliation and observability.

The network design is independent of cooling modality and rack density. Celestica DS5000 (800G) serves as the reference switch platform; Celestica DS3000 as the border leaf. Switch profiles abstract platform-specific capabilities, enabling multi-vendor operation with the same logical design. Interfaces are standard: Kubernetes APIs for intent, gNMI for switch configuration and telemetry, and OpenBMC/Redfish for hardware management context. Out-of-band management networks are not managed by Hedgehog to avoid circular dependencies.

This RA is accompanied by a companion repository [18] that provides deployable composition assets for the inference-optimized variants described in this document. Each variant folder contains a connectivity map (connectivity-map.csv) providing end-to-end device and port mapping, a topology authoring plan (topology-map.yaml), Hedgehog Kubernetes CRD wiring definitions that serve as the complete initial fabric configuration, NetBox-importable inventory, and draw.io topology diagrams. The Hedgehog wiring CRDs make literal the concept of connectivity-maps-as-code: operators can cable switches as documented, feed the provided wiring YAML into the Hedgehog fabricator tool (hhfab), and the controller will configure the entire fabric through zero-touch lifecycle management - no manual switch CLI required. The VPC API CRDs then provide a simplified interface for allocating network connectivity and tenant overlays across the running fabric. The companion repository also includes host and NIC performance tuning recommendations contributed by FarmGPU based on their production deployment. A companion repository for the AI Training Fabric RA is also available [19].

The companion repository also includes an XOC-Virtual composition that enables operators to experience the full deployment and operational lifecycle without requiring physical hardware. Hedgehog's virtual lab (VLAB) environment runs the same Hedgehog controller and Broadcom Enterprise SONiC used in production, on virtual SONiC switches. Operators can apply the

wiring CRDs from any composition in the repository to instantiate that topology in VLAB, then practice deploying, configuring, and operating the fabric - including ZTP enrollment, intent application, overlay configuration, and Day-2 reconciliation - before committing to physical infrastructure. The XOC-Virtual provides a zero-investment path to hands-on experience with the Hedgehog solution and serves as a preparation and learning environment for production deployments.

The remainder of this document presents scope, a conceptual architecture tailored to inference, overlay and multi-tenant policy design, gateway services, lifecycle operations, a decision framework for profile selection, and observability guidance.

2. Compliance with Open Compute Project Tenets

This Reference Architecture builds on established OCP work and aligns with the five OCP Tenets.

2.1. Openness

Hedgehog AI Network is fully open source (Apache 2.0) and built on SONiC, the Linux Foundation's open network operating system. All topology and overlay definitions are expressed as Kubernetes CRDs -- inspectable, extensible, and versionable in Git. Switch profiles abstract platform-specific capabilities so that operators can substitute any OCP-compliant switch while preserving the same logical design and CRD artifacts. The declarative model (CRDs, connectivity-maps-as-code) enables community contribution and modification through standard review workflows.

2.2. Efficiency

The Hedgehog controller continuously reconciles desired state to actual state on SONiC switches, eliminating CLI-driven configuration and reducing drift. Common operations -- tenant onboarding via VPC CRDs, gateway scaling, port mapping at bring-up -- use a single Kubernetes API surface. Size-tiered templates and well-defined profiles (64/128/256/512/1024

xPUs in Minimal and Distributed configurations) reduce turn-up time. Open-source software and OCP-compliant hardware with competitive supply chains keep cost structures favorable, while switch profiles preserve engineering effort across hardware generations.

2.3. Impact

By providing complete, practical artifacts -- declarative topology and overlay CRDs, connectivity-maps-as-code, gateway NAT patterns, and lifecycle workflows -- this Reference Architecture makes inference-scale OCP practices directly adoptable by organizations beyond hyperscalers. Operators start with defined size tiers and inference profiles and evolve configurations as needs change, without re-authoring device-level configurations. The alignment to OCP specifications and reliance on open software broaden accessibility and help standardize operational practices across the ecosystem.

2.4. Scale

Scaling follows the OPG-M model: 64/128/256/512/1024 xPU tiers provide non-blocking Clos [17] fabrics, and templates for each tier provide additive capacity without redesign. Tenants are added declaratively (VPC, VPCAttachment), and gateway services scale horizontally by adding servers. Because the state is declarative and continuously reconciled, expanding or modifying the fabric -- adding leaves, adjusting overlay policies, onboarding tenants -- requires no manual device touch and preserves consistency across failure and upgrade events. Management interfaces are standard (Kubernetes APIs, gNMI), enabling integration with existing tooling.

2.5. Sustainability

Right-sizing via deployment profiles and size-tiered topologies avoids over-provisioning. ZTP and continuous reconciliation reduce unnecessary site visits and configuration churn. Observability pipelines integrate with the LGTM stack (Loki, Grafana, Tempo, Mimir) to focus remediation on real issues. Open, modular software and multi-vendor hardware flexibility mean that logical designs and operational CRDs persist across hardware generations, reducing rewrite waste and extending useful service life.

3. Scope

This Reference Architecture defines an inference-optimized, multi-tenant Ethernet fabric aligned with Open Cluster Designs. It is a companion to the AI Training Fabric Reference Architecture, focusing on north-south serving, multi-tenant isolation, gateway services, and operational simplicity at the 64--1,024 xPU scale, using Hedgehog as the Kubernetes-native control plane and SONiC as the NOS on OCP-compliant switches.

The design is network-centric: topology, control plane, overlay model, telemetry, gateway services, and lifecycle workflows are specified in detail. It applies equally to air-cooled and liquid-cooled deployments; cooling modality and rack density do not affect the fabric architecture.

3.1. Cooling and Density Independence

The network architecture is identical for air-cooled (typically ~2 xPU-node servers per rack) and liquid-cooled (typically ~8 xPU-node servers per rack) deployments. The same topology, control plane, and overlay configuration apply regardless of cooling modality.

3.2. Target Compute Form Factor

This RA targets xPU-node servers -- servers containing eight xPUs (such as GPUs) where the high-bandwidth scale-up interconnect (such as NVLink) is entirely within the server chassis. Each xPU-node has scale-out NICs connecting to the Ethernet fabric. This RA treats each xPU-node as the atomic compute unit for scale-out networking.

Transceiver selection in the provided assets targets flexible-reach options (e.g., DR/SR classes) that span typical data-center layouts at the stated sizes; operators should validate optic types against measured cable lengths and, where needed, substitute DACs/AECs or alternate optics to optimize for power, cost, or reliability once rack locations and cable routes are finalized.

In scope:

- **Profiles:** Minimal Inference (no back-end; converged SO-C + storage, in-band, OoB) and Distributed Inference (adds scale-out back-end for cross-host parallelism; same converged SO-C + storage).
- **Multi-tenancy:** VPC-based per-tenant isolation with policy-controlled sharing (VPCPeering).
- **Gateway:** Hedgehog Gateway (distributed, open-source, x86) for NAT/PAT services with stateful ingress filtering; DS3000 as border leaf.
- **Size tiers:** 64, 128, 256, 512, 1,024 xPUs.
- **Control plane:** Hedgehog (Kubernetes-native), CRD-based topology and overlay definitions, continuous reconciliation to SONiC switches.
- **Lifecycle workflows:** ZTP via ONIE, Day-1 intent application, Day-2 operations and observability (Grafana Alloy to LGTM).
- **Connectivity budgets:** xPU-node NIC requirements, storage connectivity, border/external connectivity patterns.

Out of scope:

- **Scale-up** (intra-server HBN/NVLink) internals.
- **1.6T platforms** (beyond the scope of this revision).
- **OoB management control** by Hedgehog (to avoid circular dependencies; simple traditional configs assumed).
- **Facility:** Power distribution, cooling system design, and physical rack layout beyond connectivity.
- **Rack-spanning scale-up architectures:** Designs where the scale-up interconnect spans multiple racks are out of scope for the scale-up portion; the scale-out Ethernet fabric described here remains applicable for inter-node traffic.

Relationship to OCP contributions:

This RA provides the network-centric view of OPG-M (leaf-layer attachment and overlay) and XOC-N (composition and scaling tiers) for inference workloads. It assumes 8 xPUs per xPU-node, matching the OPG-M exemplar form factor. It uses SONiC as the NOS and assumes

OpenBMC/Redfish for hardware management context. Rack-scale foundations such as Open Rack version 3 (ORv3) are assumed where applicable.

This RA complements the AI Fabric (Training) RA and aligns with Open Cluster Designs, using the same open software stack (Hedgehog/SONiC) and OCP hardware foundations.

4. Architecture Overview

This section presents the architecture for multi-tenant AI inference fabrics managed by the Hedgehog Open Network Fabric. It focuses on profile-based design (Minimal vs Distributed Inference), gateway prominence for ingress/egress, and open control/data-plane roles consistent with OCP.

4.1. Declarative CRD Model

All fabric state is expressed through Kubernetes CRDs managed by the Hedgehog controller:

- **Wiring API** (`wiring.githedgehog.com`): Switch, Server, and Connection resources define physical topology -- port roles, aggregation types (unbundled, bundled/LAG, ESLAG, fabric links), and switch-to-switch/switch-to-server relationships.
- **VPC API** (`vpc.githedgehog.com`): VPC, VPCAttachment, VPCPeering, External, ExternalAttachment, and ExternalPeering resources define tenant overlays, inter-tenant connectivity, and north-south exposure.

Operators apply these CRDs with `kubectl apply` or `hhfctl`, and the Hedgehog controller reconciles the declared state to switches. The model is GitOps-compatible: CRDs are the source of truth, supporting version control, review, change control, and rollback.

4.2. Key Concepts: OPG and XOC

Two building blocks from the OCP Open Cluster Designs specifications structure every deployment:

OPG pod (Open Pod Group for M xPUs, OPG-M): A modular cluster building block consisting of xPU-node servers plus their networking, storage, and management resources. The "M" is the total xPU count -- for example, OPG-64 contains 8 eight-xPU servers (64 xPUs total). An OPG defines the leaf-layer attachment.

XOC cluster (xPU Open Cluster for N xPUs, XOC-N): A complete scale-out cluster composed of one or more OPG pods connected under a shared spine layer. The "N" is the total xPU count -- for example, a 128-xPU inference cluster composes two OPG-64 pods under a shared spine layer, yielding 128 xPUs with consistent topology and control-plane semantics. The XOC adds spine/aggregation and north-south connectivity.

Example: A site deploys a 128-xPU inference cluster by connecting two OPG-64 pods (8 xPU-node servers each, 64 xPUs each) under a shared spine layer. Gateway servers attach to border leaves for ingress/egress, and tenant overlays span both pods.

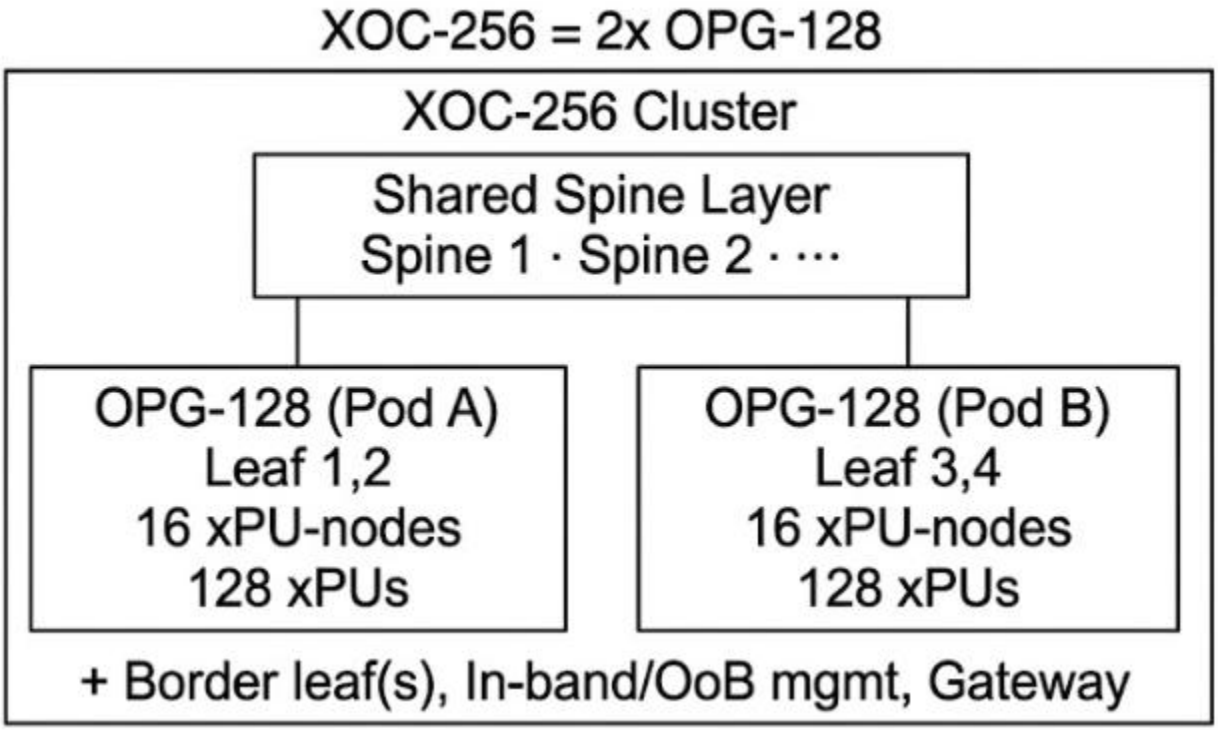

Figure 1: XOC-256 composed from two OPG-128 building blocks under a shared spine layer. Each OPG pod contains leaf switches and xPU-nodes; the XOC adds spine/aggregation and north-south connectivity.

These terms originate from the OCP Open Cluster Designs System Architectures: OPG-M System Architecture [1] and XOC-N System Architecture [2].

4.3. Structural Composition and Building Blocks

4.3.1. Network-Centric Perspective

This RA specifies the inference Ethernet fabric. Non-network components are assumed OCP-compliant and comparable to the examples presented in the OCP OPG-M and XOC-N system architecture papers, unless otherwise noted [1][2].

4.3.2. Baseline Assumptions

Parameter	Value	Notes
xPUs per server	8	xPU-node form factor
Scale-out NICs per server	8	NIC1 ... NIC8, GPU-affinitized
DS5000 total ports	64@800G, 128@400G, 256@200G	Each port supports breakout
DS5000 uplinks (to spines)	32@800G	50% of ports
DS5000 downlinks (to servers)	32@800G, 64@400G, 128@200G	50% of ports

4.3.3. OPG-M Building Blocks for Inference (Scale-out fabric)

OPG Size	Leaves	Servers (8 xPU / node)	xPUs	Notes
OPG-32	1	4	32	Hyperconverged (SO-C/Storage/Scale-out); mesh viable
OPG-64	1	8	64	Single-Homed (SH) Clos; mesh viable
OPG-128	2	16	128	Dual-plane SH Clos; Single Plane Rail-Optimized (mesh viable)
OPG-256	4	32	256	Rail-Optimized, Single & Dual Plane
OPG-512	8	64	512	Rail-Optimized, Single & Dual Plane

This example table shows leaf counts for the scale-out fabric - the complete bill of materials (BoMs) for each OPG is provided on the companion repository[18]. Leaf counts assume single-plane, 400GbE scale-out NIC configuration (one leaf serves up to 64 xPUs). Each OPG pod is

a building block, not a standalone deployment -- it requires spine/mesh connectivity and gateway/external connectivity to form a complete cluster.

4.3.4. Cluster Compositions for Inference (Scale-out fabric)

Cluster Size	Composition	Leaves	Spines	Servers	xPUs
XOC-64	2x OPG-32	2	Mesh	8	64
XOC-128	2x OPG-64/ 1x OPG-128	2	Mesh	16	128
XOC-256	4x OPG-64/ 2x OPG-128/ 1x OPG-256	4	2	32	256
XOC-512	8x OPG-64/ 4x OPG-128/ 2x OPG-256/ 1x OPG-512	8	4	64	512
XOC-1024	16x OPG-64/ 8x OPG-128/ 4x OPG-256/ 2x OPG-512	16	8	128	1024

This example table shows leaf counts for the scale-out fabric - the complete BoMs for each OPG is provided on the companion repository[18]. For smaller compositions (1-3 leaf switches), Hedgehog provides the option to support a mesh configuration in which the leaf switches directly interconnect, providing a fully non-blocking fabric without requiring separate spine switches. Hedgehog abstracts the configuration for the mesh or spine topology, providing a consistent operator interface for tenant configuration regardless of the underlying interconnect pattern.

Spine counts provide a non-blocking Clos fabric at each tier; the design reserves 50% of DS5000 ports as uplinks to spines, yielding full bisection bandwidth within the scale-out plane.

4.4. Control Plane and Data Plane

- **Control plane:** The Hedgehog controller runs in Kubernetes, watches CRDs for topology and overlays, computes desired state, and reconciles to switches via per-switch agents. Drift is detected and corrected continuously. If the controller is temporarily unavailable, switches continue forwarding with the last applied SONiC configuration; reconciliation resumes when the controller returns.
- **Data plane:** SONiC runs on switches (such as Celestica DS5000) and provides forwarding, routing, and EVPN/VXLAN overlay services.
- **Interfaces:** Kubernetes APIs for intent, gNMI for switch configuration and telemetry, observability export to LGTM, and OpenBMC/Redfish for hardware management.

4.5. Multi-Tenant Overlay Model

Multi-tenancy is the primary design driver for inference fabrics. Keep in mind that tenancy does not only refer to a 3rd party tenant - the same model applies to how infrastructure is allocated to support different teams and workload needs whether infrastructure is allocated for internal or external tenancy. The Hedgehog VPC abstraction provides per-tenant isolation by default and policy-controlled sharing where required:

- **VPC per tenant:** Each tenant maps to a VRF with one or more subnets (VNIs). Default import/export targets enforce isolation -- tenants cannot reach each other unless explicitly peered.
- **VPCAttachment:** Assigns server ports to tenant VPCs, binding the physical interface to a specific overlay segment.
- **VPCPeering:** Provides controlled inter-tenant routing where required (for example, a shared storage VPC accessed by multiple tenant VPCs).

- **External connectivity:** External, ExternalAttachment, and ExternalPeering bind VRFs to upstream networks. The Hedgehog Gateway provides NAT/PAT services for ingress/egress with stateful ingress filtering.

EVPN distributes routes across the fabric (Type-2 for MAC/IP, Type-5 for IP prefixes). Symmetric IRB provides Layer 3 forwarding across VXLAN domains. Route Distinguishers (RDs) and Route Targets (RTs) are allocated deterministically per VPC and subnet. On TH5-based switches (DS5000), all overlays use L3VNI.

4.6. Profiles and Networks

- **Minimal Inference (2 managed networks):** Converged SO-C + Storage, In-band; OoB (unmanaged by Hedgehog). No scale-out network. Right-sized for single-server inference where all models fit within a single xPU-node.
- **Distributed Inference (3 managed networks):** Scale-out (Back-end), Converged SO-C + Storage, In-band; OoB (unmanaged by Hedgehog). Adds a scale-out back-end fabric for cross-host parallelism (model sharding, pipeline parallelism, distributed batching, shared caches). Lossless RDMA transport is enabled by setting `spec.roce: true` on back-end switches.

The following network types are relevant to inference deployments:

- **Scale-out (Back-end) Fabric** (Distributed profile only): Ethernet fabric interconnecting xPU-nodes for cross-host parallelism. DS5000 switches; RDMA-capable with RoCE when enabled.
- **SO-C Network** (Scale-out Connectivity): Connectivity for CPU, gateway, and ingress services. Each xPU-node connects via BlueField-3 (or equivalent), providing 2×200G across two leaf switches. Multi-homing for front-end connectivity uses Hedgehog's HostBGP feature [20]: a host-side FRR container (`ghcr.io/githedgehog/host-bgp`) automatically establishes unnumbered eBGP sessions with redundant leaves, and the host advertises its Virtual IPs (VIPs) as /32 routes into the fabric. The fabric then uses

ECMP to load-balance and fail over across the redundant leaf paths — eliminating manual coordination, peer-link overhead, and L2 protocol complexity. HostBGP is enabled declaratively by setting `hostBGP: true` on the relevant VPC subnet. Converged with storage in both profiles.

- **Storage Network**: Connectivity for storage servers/arrays; converged with SO-C in both profiles. Multi-homing is recommended for storage resilience. HostBGP provides L3 multi-homing with automatic ECMP failover and no L2 protocol overhead; ESLAG provides L2 port-channel semantics where required.
- **In-band Management Network**: Cluster management and telemetry; managed by Hedgehog. Celestica DS2000 used in examples.
- **Out-of-band (OoB) Management Network**: BMC/hardware management; not managed by Hedgehog to avoid circular dependencies. Celestica DS1000 used in examples.

Gateway is core in both profiles. Cooling modality is orthogonal -- the same topology, control plane, and overlays apply to both air-cooled and liquid-cooled deployments.

4.7. Size-Tier Guidance

- **Primary tiers:** 64, 128, 256, 512, 1,024 xPUs. Templates for each tier provide a complete starting point with consistent control-plane semantics.
- **Additive scaling:** The same CRD abstractions scale across tiers. Gateway capacity grows independently by adding servers.

4.8. Topology-Aware Tenant Placement

Topology-aware tenant placement is one of the most effective tools for managing spine utilization in multi-tenant inference clusters. The leaf-spine Clos topology creates natural placement boundaries -- called "first-hop domains" -- that operators can exploit to keep traffic off the spine.

4.8.1. First-Hop Domains

In the single-homed Clos cabling pattern, all eight scale-out NICs from each xPU-node connect to the same leaf switch. With the DS5000 at 1x400GbE per NIC, this places 8 xPU-nodes (64 xPUs) on a common leaf, forming an 8-node first-hop domain. Within this domain, all traffic between nodes is forwarded within the leaf switch and never traverses the spine. Any tenant workload allocated entirely within a first-hop domain contributes zero spine traffic regardless of its communication pattern.

This property is particularly valuable for multi-tenant inference. Consider an inference-as-a-service provider where the majority of tenants/workloads are allocated for 1--4 xPU-nodes: if tenants/workloads are allocated within first-hop domain boundaries, those tenants produce no spine traffic. The provider can still allocate larger tenants that span multiple domains, but by keeping a significant fraction of tenancy within 8-node first-hop domains, overall spine congestion is reduced -- enabling the cluster to serve more tenants with predictable latency. A Dual-Plane Single-Homed Clos composition can increase the size of the first-hop domain to 16 nodes using a 2x400G Scale-out NIC form factor, and up to 32 nodes using a 2x200G Scale-out NIC form factor. Assets including connectivity maps and wiring diagrams for each of these composition variants is provided on the companion repository[18].

4.8.2. Placement Recommendations

- Allocate tenants within first-hop domains wherever possible. Workloads confined to a single domain produce zero spine traffic.
- Reserve cross-domain allocations for tenants that genuinely require more than 64 xPUs.
- Integrate domain awareness into the workload scheduler (for example, Kubernetes node labels). Label nodes by their first-hop domain so that the scheduler can prefer domain-local placement.
- For Distributed Inference with back-end traffic, topology-aware placement keeps both north-south and east-west traffic patterns predictable.

4.9. RoCE for Distributed Scale-out

Distributed Inference enables lossless RDMA transport via a single switch-level toggle: setting `spec.roce: true` on the Switch CRD activates PFC and ECN for RDMA traffic on that switch. QPN-aware ECMP hashing (`spec.ecmp.roceQPN`) improves load distribution for RDMA flows. Minimal Inference omits the back-end network entirely and does not use RoCE.

4.10. Host and NIC Performance Tuning

Fabric-level QoS provides the network foundation, but achieving optimal end-to-end performance also requires careful host OS and NIC configuration. Host-side tuning — including NIC firmware settings, interrupt affinity, memory registration policies, and kernel parameters — has a material impact on throughput and tail latency for both training collectives and inference serving. FarmGPU developed a highly optimized host and NIC tuning configuration during their XOC-512 B200 deployment that delivers exceptional results as confirmed by SemiAnalysis performance analysis. FarmGPU has generously shared detailed information on their NIC and host OS performance tuning approach, which is provided in the companion repository [18]. Operators are encouraged to use these tuning recommendations as a starting point and adapt them based on workload-specific observation.

4.11. Operations Alignment

- **Day 0:** ONIE-based ZTP enrolls switches; identity, Autonomous System Number (ASN), and switch profile assignment are automated. No manual CLI.
- **Day 1:** Tenants onboarded via CRDs (VPCs, attachments, externals); connectivity-maps-as-code validate topology and labels. RoCE enabled on back-end switches if the Distributed profile is selected.
- **Day 2:** Continuous reconciliation enforces intent and corrects drift. Observability via Grafana Alloy exports metrics and logs to the LGTM stack. Safe upgrades and capacity expansions follow declarative workflows.

5. Hardware Baseline

This section outlines hardware considerations for inference fabrics. Physical building blocks mirror the training RA; differences are in quantities and posture (gateway prominence, optional scale-out fabric).

5.1. Guidelines and Assumptions

- **Switches:** 800G class (e.g., Celestica DS5000) with SONiC; use Hedgehog switch profiles for multi-vendor abstraction.
- **Profiles:** Minimal (no back-end) vs Distributed (adds back-end). RDMA/PFC posture applies only to the back-end in Distributed.

5.2. Back-End Overlay Considerations (TH5)

On Broadcom Tomahawk 5 (TH5) class silicon (such as DS5000), all overlays use L3VNI -- TH5 and Broadcom Enterprise SONiC operate with a pure Layer 3 overlay model on this platform. This applies to the back-end network in Distributed Inference; Minimal Inference omits back-end overlays entirely.

5.3. Gateway Servers

- x86 servers for distributed gateway; scale by connections/s and throughput; attach to border leaves.
- Health checks and graceful drain required for rolling updates; integrate with upstream L4 load balancer (LB) or DNS.

5.4. Optics and Cabling

OCP-compliant optics; cable plans mirror the training RA with scaled counts for 64--1,024 xPUs. Maintain consistent labeling for connectivity-maps-as-code generation.

5.5. Power, Space, and Weight

Power, space, and weight scale with the selected tier. Gateway footprint is driven by connections/s and throughput targets rather than xPU count alone.

6. Topology Patterns

This section summarizes topology fundamentals and size-tier patterns for inference. Physical Clos designs mirror training; inference adds gateway prominence and the option to omit the scale-out network.

6.1. Topology Fundamentals

Leaf-spine Clos; leaves connect to xPU nodes and to spines; border leaves connect to upstream networks and gateway servers. Single-plane or dual-plane as dictated by resiliency, scale, and bandwidth needs.

6.2. Size Tier Topologies (Summaries)

- **XOC-64 (2x OPG-32):** Provides a hyper-converged mesh design with connectivity for SO-C, Storage, and Scale-out in a single meshed pair of DS5000's - ideal for minimizing device count for small/edge clusters.
- **XOC-128 (2x OPG-64 or 1x OPG-128):** The OPG-64 scale-out fabric consists of 1x DS5000 with a single-homed Clos connection pattern in which all scale-out NICs for 8 xPU servers (assuming 1x400G per NIC) form a first hop domain. The composition of 2x OPG-64 includes 16 xPU servers formed into 2x 8-node first hop domains. The composition of 1x OPG-128 includes 16 xPU servers connected 2x 51T DS5000 switches in a rail-optimized pattern in a 2-leaf rail-domain with leaf-01 handling rails 1-4 and leaf-02 handling rails 5-8. Either scale-out fabric option is provided in a mesh topology option in which the 2 scale-out leafs directly interconnect, eliminating the need for a spine layer at this scale.
- **XOC-256 (4x OPG-64, 2x OPG-128, or 1x OPG-256):** The OPG-64 and OPG-128 compositions have the benefits as detailed above, but at a larger scale. 2 OPG-256

composition options are provided for single or dual-plane rail-optimized architectures. The single plane design introduces a 4-leaf 1st-hop rail-domain where leaf-01 connects rails 1-2, leaf-02 connects rails 3-4, leaf-03 connects rails 5-6, and leaf-04 connects rails 7-8. The dual-plan design includes the same quantity of 4 leaf switches, 2 of which are allocated as plane-a and 2 are allocated as plane-b. For both planes, leaf-01 terminates rails 1-4 and leaf-02 connects rails 5-8.

- **XOC-512 (8x OPG-64, 4x OPG-128, 2xOPG-256, 1xOPG-512):** The OPG-64, OPG-128 and OPG-256 based compositions have the benefits as detailed above, but at a larger scale. The OPG-512 is offered in 2 primary variants to provide single, and dual-plane options. The single-plane option includes 8 leaf switches where each leaf switch provides connectivity for 1 rail per leaf switch - representing the largest 1st hop rail domain size for a single plane design assuming 1x400GbE scale-out NIC and 51T switch form factors. The dual-plane option consists of the same quantity of 8 leafs broken into 2 planes with 4 switches each. For either plane, each switch connects 2 rails, leaf-01 connects rails 1-2, leaf-02 connects rails 3-4, leaf-03 connects rails 5-6, and leaf-04 connects rails 7-8.
- **XOC-1024 (16x OPG-64 or 8x OPG-128, 4x OPG-256, 2x OPG-512, 1x OPG-1024):** The OPG-64, OPG-128, OPG-256 and OPG-512 based compositions have the benefits as detailed above, but at a larger scale. The OPG-1024 based composition introduces a dual-plane, 16-leaf composition with 8 leafs per plane. For each plane, a single leaf switch is provisioned per-rail. This is the largest 1st-hop rail domain size achievable with 400G (2x200G) per scale-out NIC.

Note: Detailed port maps and Bills of Materials (BoMs) follow training RA patterns with scaled counts; connectivity-maps-as-code ensures label consistency.

6.3. Per-Profile Topology

- **Minimal Inference (no back-end):** single Clos for converged SO-C + storage; in-band present; OoB unmanaged. Gateway servers connect to border leaves.

- **Distributed Inference (with back-end):** second Clos (or logical network) for RDMA back-end; RoCE enabled on back-end switches; SO-C + storage posture identical to Minimal (converged).

6.4. Gateway Placement

- Attach gateway servers to the border leaf pair (L3) with sufficient bandwidth for connections/s and throughput targets.
- Distribute Virtual IPs (VIPs) via upstream Layer 4 (L4) load balancer or DNS; ensure health checks and graceful drain for rolling changes.
- Keep stateful NAT at the gateway; fabric remains L3-only for predictability. Use upstream firewall/Access Control Lists (ACL) where explicit policy is required.

7. Overlay and Multi-Tenancy

This section describes how logical overlays and policies are realized using the Hedgehog VPC abstraction, with EVPN for control plane and VXLAN for data plane. Multi-tenancy is the primary concern for inference: per-tenant isolation, controlled sharing, and gateway integration are the design drivers.

7.1. Overlay Model (VPC to VRF/VNI)

7.1.1. Abstractions

- VPC maps to a VRF (tenant routing context).
- VPC subnets map to VNIs (one per segment), with symmetric IRB for routing.
- Route Distinguishers (RDs) and Route Targets (RTs) are allocated deterministically per VPC and subnet.

7.1.2. Realization

- EVPN Type-2 routes advertise MAC/IP reachability for bridged endpoints.
- EVPN Type-5 routes advertise IP prefixes for routed segments and external connectivity.
- BGP distributes EVPN routes across the fabric; SONiC applies data-plane state.

- On TH5-based leaves (DS5000), all overlays use L3VNI due to chipset constraints (see Section 5.2).

7.1.3. VPC API Resources and Policy Control

Resources:

- VPC defines the VRF and default import/export policy.
- VPCAttachment binds server interfaces to VPCs with tagging.
- External, ExternalAttachment, ExternalPeering define north-south connectivity and BGP peers.
- VPCPeering optionally enables routed connectivity between VRFs with policy controls.

Policy:

- Import/export targets per VPC enforce isolation by default and allow scoped connectivity (for example, selective inter-VPC peering).
- Route policies control redistribution to externals (for example, advertise only Type-5 routes to upstream peers).
- Default policies are safe and minimal; tuned profiles can be provided for specific deployment needs.

Gateway integration: External exposure uses External/ExternalAttachment/ExternalPeering. The Hedgehog Gateway provides NAT/PAT services for ingress/egress and scales by adding servers. Implicit ingress filtering is available via stateful NAT; explicit firewall/ACL enforcement should be handled by external components.

7.2. EVPN/VXLAN Realization

- **Control plane:** EVPN Type-2 for MAC/IP advertisement (bridging), EVPN Type-5 for IP prefixes (routed). Symmetric IRB provides L3 forwarding across VXLAN domains.

- **Data plane:** VXLAN encapsulation; ECMP ensures path diversity for low-variance latency. Deterministic RD/RT allocation per VPC/subnet.
- **TH5 consideration:** TH5-based switches (DS5000) use L3VNI for all overlays. Minimal Inference omits back-end overlays entirely. In rail-optimized underlays, rail-local flows remain at the leaf; cross-rail flows traverse spines. Rail-optimized is a wiring constraint; switch counts are unchanged vs standard Clos.

7.3. Per-Network Policy

- **SO-C:** Emphasize north-south policy and gateway integration; apply explicit allow/deny policies using upstream ACL/firewall components where required.
- **Storage:** Segregate from tenant SO-C traffic where separate; conservative external redistribution. Multi-homing is recommended for storage resilience; HostBGP provides L3 multi-homing with automatic ECMP failover; ESLAG provides L2 port-channel semantics where required.
- **Back-end (Distributed only):** Enable RoCE selectively; QPN-aware hashing (`spec.ecmp.roceQPN`) improves load distribution for RDMA flows. Multi-homing is not required for collective traffic in most deployments because each xPU-node NIC connects to a single leaf.
- **In-band:** Protect telemetry/control; rate-limit; isolate from tenant traffic.

7.4. Traffic Forwarding

All SO-C, storage, and in-band traffic uses standard SONiC forwarding, which provides effective 5-tuple ECMP hashing for low-variance latency distribution across leaf-spine paths. For Distributed Inference, enabling RoCE on back-end switches adds lossless RDMA transport with PFC and ECN (see Section 11.3 for configuration details).

8. Gateway and External Connectivity

Inference clusters are predominantly north-south: client requests enter, responses exit. The gateway is therefore a crucial consideration in inference infrastructure design. This section

describes the distributed gateway architecture, external connectivity resources, ingress/egress patterns, NAT/PAT and firewall policy, per-tenant controls, scaling/HA, and integration with upstream load balancers.

8.1. Hedgehog Gateway Architecture

- **Distributed x86 instances;** each maintains local connection state (Network Address Translation (NAT) conntrack) but requires no cross-instance shared state. No single appliance chokepoint.
- **Open-source implementation;** Kubernetes-native management via Custom Resource Definitions (CRDs).
- **Scale-out** by adding servers to the gateway pool; capacity planning by connections/s and latency.
- **Connectivity:** Gateway servers attach to the SO-C fabric to provide policy controlled connectivity directly between VPC's as well as between VPC's and external resources.

8.2. External Connectivity and Gateway CRDs

Operators configure external exposure declaratively via VPC/External resources and express gateway services via the Gateway API:

- `External`: defines an external connectivity point.
- `ExternalAttachment`: binds tenant VPCs (VRFs) to an `External`.
- `ExternalPeering`: configures BGP peering for route exchange when applicable.

Gateway services are configured via `gateway.githedgehog.com/v1alpha1`:

- `Gateway`: defines a gateway node (interfaces, BGP neighbors, VTEP, ASN, workers, etc.).
- `GatewayGroup`: groups gateway instances; `Peering` targets a group via `spec.gatewayGroup`.
- `Peering`: inserts gateway services between VPCs or between a VPC and an external; NAT can be configured per expose entry.

NAT configuration (per `Peering` expose entry, `spec.peering[<vpc>].expose[].nat`):

- `masquerade`: stateful source NAT with connection tracking; supports `idleTimeout` (default 2 min) for flow removal. Functions as a simple firewall by dropping packets without established state.
- `portForward`: stateful destination NAT for specific port mappings (protocol, port, target); also supports `idleTimeout`.
- `static`: stateless one-to-one address mapping between `ips` and `as` CIDRs; no per-flow state.

What operators specify:
External endpoints and VRF bindings (via External/ExternalAttachment/ExternalPeering); NAT mode and CIDR mappings (via Peering expose entries).

What the controller renders:
Route programming, policy enforcement on gateways, VRF-consistent forwarding and return path behavior.

8.3. Ingress Patterns

Typical ingress for inference request traffic:

- Client connects to external Virtual IP (VIP), public or private.
- Upstream L4 load balancer distributes to gateway instances (or DNS across gateways).
- Gateway applies implicit ingress filtering via stateful NAT (packets without established state are dropped); `portForward` DNAT if exposing internal service IPs.
- Packet routed through fabric to destination server in tenant VRF via EVPN/VXLAN.
- Application handles request; response follows established state.

Multi-tenant considerations: Per-tenant external IP ranges and policies; isolation by VRF with default-deny posture. Logging/metrics labeled by tenant for audit and SLOs.

Protocols: HTTP/HTTPS, gRPC, and custom RPCs; TLS termination strategy is deployment-specific.

8.4. Egress Patterns

Outbound traffic (model fetches, telemetry export, external APIs):
`masquerade` SNAT to tenant or shared pools; scope controlled by `Peering` expose CIDRs. Return traffic follows NAT connection state; observability captures errors/timeouts for troubleshooting.
Note: explicit egress allow/deny lists and per-tenant rate limits require supplemental controls (external firewall or future Gateway API extensions) beyond what the current `Peering` resource provides.

8.5. NAT/PAT Configuration

- Source NAT (SNAT) via `masquerade`: outbound translation from tenant IP space to external pools with connection tracking; scope per VPC/tenant where required. Supports `idleTimeout` (default 2 min).
- Destination NAT (DNAT) via `portForward`: inbound exposure of services via external VIPs to internal service IPs on specific ports/protocols; also stateful with `idleTimeout`.
- Static NAT via `static`: one-to-one address mapping between `ips` and `as` CIDRs without per-flow state; suitable for fixed address translation.
- Declarative approach: all NAT configuration is expressed per expose entry in Gateway `Peering` resources; no manual device CLI.

8.6. Firewall Policies

- **Implicit filtering:** stateful NAT (masquerade/portForward) drops packets without an established connection state, providing default-deny behavior for ingress.
- **Scope control:** expose entries in `Peering` resources define which CIDRs and ports are reachable per VPC, limiting the attack surface.

- **Limitations:** the current Gateway API does not expose dedicated firewalls or ACL CRDs. Operators requiring fine-grained match criteria (source/destination prefixes, protocols, rate-limits) or layered global/per-tenant rule hierarchies should supplement with external firewall appliances or await future Gateway API extensions.

8.7. Per-Tenant Gateway Policies

- **Isolation:** per-tenant routing (VRF) and policy boundaries; cross-tenant blocked by default.
- **Capabilities:** per-tenant NAT configuration (masquerade, portForward, static per Peering expose entry); scope control via CIDR selection. Logging and metrics are scoped per tenant where the observability stack supports it.
- **Explicit policy:** fine-grained allow/deny policies use external firewall or ACL components alongside the gateway.

8.8. Scaling and High Availability

- **Horizontal scaling:** add gateway instances; each instance is independently stateful (local conntrack only), enabling linear capacity growth for connections/s and throughput.
- **High Availability (HA):** health checks remove unhealthy instances; rolling updates with graceful drain preserve availability.
- **Capacity signals:** sustained high connections/s, CPU, or added latency on new connections; scale thresholds defined operationally.

8.9. Load Balancer Integration

Upstream L4 load balancer (LB) or DNS distributes traffic across gateway instances.
Health checks: HTTP/TCP endpoints report readiness/liveness; LB evicts failed nodes.
Operational flows: drain before maintenance; coordinate LB timeouts with gateway tracking timers.

8.10. Operational Guidance and SLO Alignment

Measure and alert on: connections/s, handshake latency, per-tenant connection counts, NAT table utilization, errors, CPU/mem.

Keep stateful functions at the gateway; keep the fabric L3-only for predictability and simplicity. See Appendix B for SLI recommendations.

9. Lifecycle and Automation

This section describes Day 0/Day 1/Day 2 operations for inference fabrics. It reuses the training lifecycle model while adding tenant onboarding patterns.

9.1. Day 0: Initial Deployment

Discovery via ONIE

Switches ship with ONIE (Open Network Install Environment), an open-source boot loader for bare-metal switches originating from the Open Compute Project. On power-on, ONIE enters discovery mode and requests network configuration via DHCP. In this RA, DHCP options provide the location of the Hedgehog controller/installer so that switches can enroll automatically without manual configuration. ONIE attribution: ONIE is an OCP project; Hedgehog works with ONIE to automate switch onboarding.

Zero-Touch Provisioning (ZTP) Workflow

The Zero-Touch Provisioning workflow removes the need for device-level CLI and templates. The sequence is:

- Switch boots into ONIE and obtains network parameters via DHCP.
- DHCP response includes the Hedgehog controller/installer endpoint.
- The switch downloads and installs the Hedgehog agent and the base SONiC image appropriate for its platform.
- The agent registers with the Hedgehog controller and presents its identity and platform characteristics.

- The controller assigns switch identity, Autonomous System Number (ASN) and management parameters, and binds a switch profile that encodes platform-specific capabilities (for example, port layout, speed/FEC, breakout).
- The agent configures SONiC accordingly and establishes management channels (for example, gNMI for configuration/telemetry).
- The switch appears as a managed resource and is ready for intent application.

Key properties:

- No manual CLI on switches; identity and keys are provisioned automatically.
- Switch profiles abstract hardware specifics, enabling multi-vendor operation with the same logical design.
- The outcome is a SONiC-based switch that acts as a data-plane element controlled by the Hedgehog controller, ready to participate in the fabric.

Initial Verification

Upon successful registration, the controller confirms switch reachability and basic health. Minimal checks include agent heartbeat, gNMI readiness, and platform profile application. At this point, operators can list inventory and status using standard Kubernetes tools (for example, kubectl get against CRDs) or the Hedgehog CLI (hhfctl). The fabric is now ready for Day-1 intent.

9.2. Day 1: Configuration (Tenant Onboarding)

Intent Application

Operators apply Wiring API CRDs (`Switch`, `Server`, `Connection`) and VPC API CRDs for tenants:

Create per-tenant `VPC` (VRF) and subnets; define `External` if north–south exposure is required.

Attach servers to VPCs via `VPCAttachment`; define `ExternalAttachment` and `ExternalPeering` as needed.

For Distributed Inference, define back-end networks and enable RoCE if required.

RoCE Enablement (Distributed Inference Only)

For Distributed Inference deployments with a back-end RDMA network, enable lossless transport on the relevant switches by setting `spec.roce: true` in the Switch CRD. This activates PFC and ECN for RDMA traffic automatically.

QoS and Hashing Baseline

Apply DSCP→queue mapping (Appendix A) fabric-wide; configure server-facing ports to trust host DSCP markings.

Enable ECN on front-end/storage classes with conservative thresholds; keep PFC off except for the RDMA class when back-end exists.

Set and verify consistent ECMP hashing seeds across leaves/spines; record in inventory.

Validation

Admission/webhook checks enforce schema and semantics; connectivity-maps-as-code validate port/NIC mappings and tenant boundaries. A connectivity-map generator derives expected switch-to-switch and server-to-leaf mappings and can be used to validate cabling plans and port/NIC mapping prior to physical turn-up.

Rollout Patterns

- Canary new tenants or changes; sequence updates; rollback via git; reconciliation enforces desired state.
- Safe operations: drain traffic from affected attachments where appropriate (for example, when changing connection types) to minimize disruption during changes.

GitOps Compatibility

Operators store CRD artifacts as versioned documents (YAML) and apply them in logical units (for example, per-tier or per-tenant). The controller computes the desired state and the agents

render the corresponding SONiC configuration on the switches. This process is compatible with GitOps: the CRDs serve as the source of truth, supporting review, change control, and rollback.

9.3. Day 2: Operations

Observability

Grafana Alloy collects metrics and logs from switches and agents via gNMI streaming telemetry and exports to the LGTM stack (Loki, Grafana, Tempo, Mimir). Standard interfaces are used end-to-end -- Kubernetes APIs, gNMI, Prometheus exposition, and Redfish/OpenBMC for hardware inventory -- enabling integration with existing enterprise tooling. See Section 11 for the full observability pipeline and SLI catalog.

Continuous Reconciliation and Drift Management

The Hedgehog controller continuously compares desired state (Kubernetes CRDs) with actual state observed from switches. Deviations (for example, accidental configuration drift, link role changes, or component replacement) are detected and corrected by reconciliation. When operators commit changes to Kubernetes CRDs, reconciliation computes the minimal delta and applies it safely through the agents. If the Hedgehog controller is temporarily unavailable, switches continue forwarding with the last applied SONiC configuration; reconciliation resumes when the controller returns.

Failure Handling

Hedgehog agents report health and status to the controller, including switch reachability and key component metrics. The controller flags anomalies (for example, a leaf becomes unreachable or loses upstream links) and provides context to observability pipelines for alerting and triage. Recovery procedures include replacing failed hardware and reassigning identity so that the same logical configuration re-applies, or isolating impacted attachments while maintaining fabric integrity. Because intent is authoritative, replacement workflows are deterministic and largely hands-off beyond physical swap.

Maintenance Operations

Common Day-2 operations are coordinated through the controller using declarative inputs:

- Switch OS upgrades: roll SONiC image updates in a controlled sequence aligned to topologies and maintenance windows.
- Agent updates: update Hedgehog agent versions centrally and verify health gates before proceeding cluster-wide.
- Configuration changes: modify CRDs for overlays, policies, or topology; controller orchestrates safe application.
- Capacity expansion: add new switches or gateway servers, apply Switch and Connection resources, and observe reconciliation; add spines/leaves per size-tier templates.
- Hardware replacement: replace a failed switch, associate prior identity/profile, and allow reconciliation to repopulate configuration.

Day-2 processes emphasize low-risk, observable changes backed by declarative state and automated reconciliation, aligning with reliability goals for inference workloads.

Multi-Rack Operational Considerations

For larger Distributed deployments spanning multiple rack groups, stage operational workflows per rack group or leaf group. For example, roll out upgrades one leaf group at a time, validate health and traffic, then proceed to the next group. Treat rack group boundaries as failure domains at the leaf layer and plan change windows accordingly.

Monitoring and Drift Detection

Track gateway KPIs (connections/s, handshake latency, NAT table utilization) during policy changes and rolling updates. Monitor per-link utilization for load-balancing health. Reconciliation corrects configuration drift automatically; observability pipelines surface any anomalies that warrant operator attention.

10. Profile Selection

This section provides a decision framework to choose between Minimal Inference and Distributed Inference profiles. The goal is to avoid over- or under-provisioning by matching fabric complexity to workload and operational needs.

10.1. Decision Flow (Textual)

1) Does inference require cross-host communication?

If ANY of the following apply, choose **Distributed Inference**:

- Model sharding or tensor/pipeline parallelism across hosts
- Distributed batching or shared caches across hosts
- Cross-host state or KV store dependencies

If ALL of the following apply, choose **Minimal Inference**:

- Single-server inference handles all models
- Predominantly north–south traffic (request/response)
- No cross-host data dependency on latency-critical paths

2) What are the operational priorities?

Choose Minimal if: operational simplicity and cost minimization outweigh flexibility; team has limited network operations expertise.

Choose Distributed if: growth to larger models is anticipated; flexibility is valued; team has or will build network operations capability.

10.2. Right-Sizing Guidance

Adopt a mix of profiles per site as needed. Track demand distributions (e.g., percentage of requests requiring >1 host). Prefer many small/medium clusters for placement advantages; expand Distributed when cross-host paths dominate SLOs. Gateway capacity is decoupled and scales independently by adding servers.

Date: April 2026

10.3. Profile-Specific Network Posture

- **Minimal Inference:** no scale-out fabric; 5-tuple ECMP hashing for good flow distribution across leaf-spine paths.
- **Distributed Inference:** back-end fabric present; one-parameter RoCE to enable PFC+ECN on the RDMA class only; front-end posture identical to Minimal (including ECMP seed verification).

Choose Minimal if:
Single-server inference handles all models, north–south traffic dominates, and operational simplicity/cost minimization are priorities.

Choose Distributed if:
Cross-host parallelism or shared state is required (sharding, pipeline, distributed batching/caches), or growth to larger models is anticipated.

Once a profile is selected and the fabric is operational, operators need visibility into fabric health and tenant-level performance. The following section covers the observability pipeline and SLO enablement.

11. SLO Enablement and Observability

This section describes how the observability pipeline enables operators to set and meet SLOs for inference workloads. The architecture exports SLIs; operators define SLO targets in their own observability systems.

11.1. SLI vs SLO

- **SLI (Service Level Indicator):** A quantifiable measure such as p99 latency, error rate, or throughput. The fabric and observability stack measure SLIs.
- **SLO (Service Level Objective):** A target for an SLI (for example, p99 < 50 ms). Operators define SLOs based on business needs.

The architecture exports SLIs with appropriate labels (per-tenant, per-device, per-port) and provides dashboards; it does not prescribe targets. Operators build SLO alerting in their own observability systems using the exported metrics.

11.2. Observability Pipeline

Telemetry collection uses Alloy, Grafana's OpenTelemetry collector, to ingest device metrics via gNMI and export to the LGTM stack -- Loki (logs), Grafana (dashboards and visualization), Tempo (traces), Mimir (metrics, Prometheus-compatible). Prometheus endpoints are exposed where applicable; metric names and labels are standardized for cross-device querying.

Key SLIs exported by the fabric include:

- **Link and port utilization** (per-device, per-port) for capacity monitoring and load-balancing health.
- **Gateway KPIs:** connections/s, handshake latency, NAT table utilization, error rates, and per-instance CPU/memory.
- **BGP session health:** session count, flaps, and convergence timing.
- **Device health:** temperature, PSU, fan status, and platform inventory.

When Distributed Inference is deployed with RoCE enabled, additional SLIs include PFC pause events and RDMA queue utilization on back-end switches.

See Appendix B for the full SLI catalog and collection methods.

11.3. One-Parameter RoCE Enablement

- **Concept:** Enable RoCE for Distributed Inference back-ends with a single toggle at fabric/VPC/profile scope; Hedgehog renders PFC/ECN/queue/hashing automatically.
- **When to enable:** Distributed Inference back-end only; not for Minimal Inference.
- **Host requirement:** Hosts must mark RDMA flows with the designated DSCP to receive lossless treatment.

12. Validation Plan

With the architecture, profiles, and observability pipeline defined, this section outlines how to validate an inference fabric deployment -- focusing on isolation, latency, and convergence. Field deployments and external reports inform performance expectations.

12.1. Objectives

- Validate tenant isolation and overlay correctness (VRF boundaries, import/export behavior).
- Confirm low-variance latency under representative loads (burst/micro-batch) and predictable ECMP behavior.
- Verify ZTP, intent application, and reconciliation; exercise failure handling (link/node/gateway).
- Ensure observability coverage (tenant-scoped SLIs; dashboards; alerts).

12.2. Methods

- **Logical checks:** CRD schema/admission validation; connectivity-maps-as-code for topology and tenant boundaries; policy linting.
- **Lab emulation:** synthetic and inference-like traffic; measure p95/p99 latency and convergence timings.
- **Virtual lab (VLAB):** Virtual lab (VLAB): Hedgehog provides a virtual lab environment that runs the same Hedgehog controller and Broadcom Enterprise SONiC (via virtual SONiC switches) used in production deployments. Operators can apply the wiring CRDs from any composition in the companion repository[18] to instantiate that topology in VLAB - the same CRDs that would configure a physical fabric are used unchanged in the virtual environment. This enables operators to validate topology, overlay, and policy configuration, practice the full ZTP-to-Day-2 lifecycle, and gain hands-on experience operating the Hedgehog solution before committing to physical infrastructure. The companion repository includes a dedicated XOC-Virtual composition designed for this

purpose: it provides virtual servers with a comparable network configuration so operators can experience the complete process of applying composition assets to a fabric, from initial switch enrollment through tenant overlay configuration. VLAB uses SONiC virtual switches that provide broad command coverage (~95%) without requiring physical switch hardware. Virtual switch port counts may differ from physical platforms, breakout emulation is not supported, a single fabric can be modeled per instance, and performance and scale are constrained by the virtualized environment. VLAB is well-suited for validating configuration correctness, CRD workflows, reconciliation behavior, failure drills, and operator training — it is not intended for comprehensive emulation or performance benchmarking.

- **Failure drills:** Link/node/gateway failures; controller/agent restarts; upgrades; verify deterministic recovery.
- **Observability:** Establish dashboards for health, capacity, per-tenant SLIs, and change monitoring; validate alerts for material failure modes including gateway degradation.

Additional inference-specific tests:

- **SO-C traffic patterns:** emulate bursty short flows representative of inference request traffic; measure p95/p99 latency under load.
- **Gateway scaling/HA:** step-load tests on connections/s and handshake latency; validate LB health checks, drain behavior, and failure recovery.
- **Scale-out RDMA (if present):** verify lossless class isolation, bounded pause behavior, and latency stability under load.

12.3. Success Criteria

- Isolation holds by default; only explicit policies permit sharing.
- EVPN routes (Type-2/5) present as expected; redistribution policies followed.
- ZTP and intent apply without manual CLI; drift corrected by reconciliation within acceptable time windows.
- Latency targets are achievable within profiled loads; deviations are explainable.

- ECMP skew remains within operational thresholds under representative short-flow mixes.
- Gateway maintains targets for connections/s and handshake latency during rolling updates.

12.4. Scope and References

- Lab validation provides partial/limited coverage; larger sizes follow the same topology and CRD patterns and scale additively.
- The Hedgehog platform and fabric design have been validated in production (including deployment at FarmGPU, validated by SemiAnalysis); the inference-specific variants in this RA extend that foundation. External reports inform performance expectations at scale.

12.5. Packaging

- Provide sample CRDs, policy examples, connectivity-map rules, validation scripts, and runbooks.
- Store inputs/outputs alongside intent artifacts (GitOps) for auditability.
- Document step-by-step procedures for Day 0/1/2 tests and failure drills; include pass/fail gates and rollback guidance.

13. Conclusion

This Reference Architecture provides a practical, multi-tenant blueprint for AI inference fabrics using Hedgehog as the Kubernetes-native control plane and SONiC as the data-plane NOS on OCP-compliant switches such as Celestica DS5000. It specifies profile-based designs (Minimal Inference for simple north-south serving, Distributed Inference for cross-host parallelism with lossless RDMA transport), per-tenant overlays via EVPN/VXLAN with Hedgehog VPC abstractions, gateway-centric ingress/egress with distributed NAT/PAT services, and observability pipelines for SLO enablement.

Date: April 2026

By defining size-tiered templates, the design scales from 64 to 1,024+ xPUs with additive changes and consistent operational practices. Continuous reconciliation, connectivity-maps-as-code, and Kubernetes CRD-driven operations reduce operational risk. The open, multi-vendor approach -- realized via switch profiles -- enables adopters to preserve designs across hardware generations within the OCP ecosystem.

The intent is to make inference-scale OCP practices broadly adoptable with clarity, precision, and practical guidance for real deployments.

14. References

[1] OPG-M System Architecture, Open Compute Project, 14 January 2026. https://www.opencompute.org/documents/opg-m-system-architecture-final-14-january-2026-pdf

[2] XOC-N System Architecture, Open Compute Project, 14 January 2026. https://www.opencompute.org/documents/xoc-n-system-architecture-final-14-january-2026-pdf

[3] Software for Open Networking in the Cloud (SONiC), Linux Foundation. https://sonicfoundation.dev/

[4] Celestica DS5000 64x800G Switch Specification, OCP Contribution. https://www.opencompute.org/documents/celestica-ds5000-64x800gb-switch-specification1-2-pdf

[5] Celestica DS3000 Series. https://cls.celestica.com/hardware-platforms/ds3000/

[6] Celestica DS1000 Ethernet Access Switch Specification, OCP Contribution. https://www.opencompute.org/documents/ds1000ethernetaccesswitch-docx-pdf

[7] Hedgehog (Apache 2.0). https://githedgehog.com/

Date: April 2026

[8] Open Rack v3 Specification, Open Compute Project. https://www.opencompute.org/documents/open-rack-v3-spec

[9] ONIE -- Open Network Install Environment, Open Compute Project. https://opencomputeproject.github.io/onie/

[10] SemiAnalysis, InferenceX (formerly InferenceMAX). https://inferencex.semianalysis.com/

[11] MLPerf Inference (scenarios and benchmarks). https://github.com/mlcommons/inference

[12] SONiC QoS Configuration Guide. https://github.com/sonic-net/SONiC/wiki/QoS

[13] Zhu, Y. et al., 'Congestion Control for Large-Scale RDMA Deployments' (DCQCN), ACM SIGCOMM 2015.

[14] Hedgehog Docs, External Peering. https://docs.hedgehog.cloud/latest/user-guide/external/

[15] Hedgehog Docs, Gateway User Guide. https://docs.hedgehog.cloud/latest/user-guide/gateway/

[16] Hedgehog Docs, Gateway API Reference. https://docs.hedgehog.cloud/latest/reference/gateway-api/

[17] Charles Clos., A Study of Non-Blocking Switching Networks. https://ieeexplore.ieee.org/document/6770468

[18] Open Cluster Designs Aligned AI Inference Fabric — Companion Repository, Open Compute Project. https://github.com/opencomputeproject/OCP-OCDAI-inference-fabric

Date: April 2026

[19] Open Cluster Designs Aligned AI Training Fabric — Companion Repository, Open Compute Project.
https://github.com/opencomputeproject/OCP-OCDAI-training-fabric

[20] Hedgehog Docs, Host Settings and HostBGP Container.
https://docs.hedgehog.cloud/latest/user-guide/host-settings/

15. License

16. About Open Compute Foundation

The Open Compute Project (OCP) Foundation is a collaborative community focused on redesigning hardware technology to efficiently support the growing demands on compute infrastructure. OCP shares open specifications and best practices for data center products, including servers, storage, networking, and racks, enabling faster innovation and broader choice through open ecosystems. For more information, visit https://www.opencompute.org/.

Date: April 2026

17. Glossary

Following are key terms used in this document.

Term	Definition
ACL	Access Control Lists for security enforcements.
Alloy	Grafana's OpenTelemetry collector used for telemetry ingestion.
ASN	The Switch BGP Autonomous System Number
BGP	Border Gateway Protocol used for EVPN routes, external peering.
BlueField-3	NVIDIA DPU used as the reference SO-C/Storage NIC; provides 2x200G per xPU-node.
BoM	Bill of Materials that include switches, optics, cables or gateways with quantities and specifications.
Clos	A non-blocking, multi-stage leaf-spine network topology commonly used in data centers; provides high bisectional bandwidth and predictable performance.
Custom Resource Definition (CRD)	Resource describing links between endpoints; supports unbundled, LAG, ESLAG, and fabric links.
Connectivity Map	Generated mapping of physical/logical connectivity used for validation and labeling.
ConnectX	NVIDIA network interface controller family used as the reference scale-out NIC. ConnectX-7 provides 1x400G per port.
Continuous reconciliation	The control-plane loop executed by the Hedgehog controller that constantly monitors the network to ensure actual state matches intended desired state.
Distributed Inference	Deployment profile with scale-out back-end network for cross-host parallelism (model sharding, distributed

Term	Definition
	batching, shared caches); converged SO-C + Storage, In-band, OoB.
DACs/AECs	Direct Attach Cables / Active Electrical Cables (cables types used to substitute optics)
DNS	Domain Name System
DSCP	Differentiated Services Code Point; six-bit field in the IP header used for classifying and managing network traffic, allowing hosts to select QoS class.
Dual-plane	Network design using two isolated redundant fabrics (Plane A and Plane B); each GPU-affinitized NIC has two ports (one per plane).
ECMP	Equal-Cost Multi-Path; hashing across equal-cost routes to distribute flows.
ECN	Explicit Congestion Notification; network devices signal impending congestion to end-hosts by marking the IP header, allowing proactive rate reduction.
ESLAG	Ethernet Segment LAG; EVPN-based multi-homing with DF election for switch redundancy.
EVPN	BGP control plane for VXLAN (Type-2 MAC/IP routes, Type-5 IP prefixes).
External	Logical external network definition for north-south connectivity.
ExternalAttachment	Attaches a VPC to an External network.
ExternalPeering	Configures BGP peering to an external network.
First-hop domain	The set of xPU-nodes whose scale-out NICs all connect to a common leaf switch; all traffic between nodes within a

Term	Definition
	first-hop domain is leaf-local and never traverses the spine.
GitOps	Operational model where Kubernetes CRDs are versioned in git; changes are reviewed, auditable, and reconciled.
gNMI	gRPC Network Management Interface; standard interface for device configuration, telemetry, and management.
HBN	High Bandwidth Network/Interconnect.
Hedgehog agent	Software running on each managed switch that applies state to SONiC and reports status back to the controller.
Hedgehog controller	Core component of the Hedgehog control plane running within Kubernetes; calculates desired network state and continuously reconciles actual device state to match.
Hedgehog gateway	Distributed, open-source border connectivity component running on x86 servers; provides policy-driven NAT/PAT services for north-south traffic. Managed via Kubernetes-native APIs; scales horizontally by adding servers.
Hedgehog Open Network Fabric (Hedgehog)	Kubernetes-native fabric management platform (Apache 2.0) that manages SONiC-based switches via Kubernetes CRDs and continuous reconciliation.
hhfctl	Hedgehog CLI (kubectl plugin) for fabric resource operations.
HostBGP	Hedgehog's Layer 3 multi-homing feature for frontend and storage networks. A host-side FRR container establishes unnumbered eBGP sessions with redundant leaf switches; the host advertises Virtual IPs (VIPs) as /32 routes, and the fabric provides ECMP-based load balancing and failover. Enabled by setting `hostBGP: true` on the VPC

Term	Definition
	subnet CRD. HostBGP subnets attach to unbundled connections only. See Hedgehog Docs [20].
IRB (symmetric)	Symmetric Integrated Routing and Bridging; both Layer 2 bridging and Layer 3 routing functions are performed on all edge devices (VTEPs) in the VXLAN fabric, ensuring consistent forwarding.
Kubernetes-native	Architecture built on Kubernetes primitives (Custom Resource Definitions, operators, controllers). Distinguished from 'cloud-native,' which refers to broader ecosystem patterns.
L3VNI	Layer 3 VXLAN Network Identifier used for routed overlays. TH5-based switches (such as DS5000) use L3VNI for all overlay traffic.
LGTM	Observability stack: Loki (logs), Grafana (dashboards), Tempo (traces), Mimir (metrics).
Minimal Inference	Deployment profile without back-end network; converged SO-C + Storage, In-band, OoB. Optimized for single-server serving and north-south traffic patterns.
NIC	Network Interface Card; a server component used for network connectivity.
Non-blocking (Clos fabric)	A fabric where sufficient uplinks are reserved to provide full bisection bandwidth within the scale-out plane.
ONIE	Open Network Install Environment; OCP open-source boot loader for switch discovery and imaging.
OoB	Out of Band (used for management network)
OpenBMC	Open-source baseboard management framework for platform management.

Term	Definition
OPG-M	Open Pod Group for M xPUs; a modular cluster building block (pod group) consisting of xPU-nodes plus networking, storage, and management resources. Parameterized by total xPU count 'M'.
ORv3	Open Rack v3, a foundational rack-scale standard from the Open Compute Project.
PAT	Port Address Translation (part of NAT/PAT services)
PFC	Priority Flow Control (IEEE 802.1Qbb); link-level flow control applied selectively to specific traffic classes, enabling lossless transport for protocols like RoCE.
Rail	NIC ports sharing GPU-affinity index across servers. In an 8-xPU-per-server architecture, there are 8 rails (rail-1 through rail-8).
Rail-optimized	Wiring constraint preserving rail locality with the same switch counts as standard Clos. Ports of a given rail connect to the same leaf or contiguous leaf set.
RDMA	Remote Direct Memory Access used for lossless transport
Redfish	DMTF standard API for hardware management/inventory.
RoCE	Remote Direct Memory Access (RDMA) over Converged Ethernet; requires lossless transport and ECMP-friendly hashing. Used in Distributed Inference back-ends when enabled.
Scale-out domain	The Ethernet fabric used to interconnect multiple xPU-nodes across the cluster. This domain is the primary subject of this reference architecture.

Term	Definition
Scale-up domain	High-bandwidth interconnect within a server chassis (for example, NVLink, NVSwitch); assumed contained within the xPU-node enclosure.
Server (CRD)	Resource describing a server connected to the fabric.
Single-plane	Network design using a single unified scale-out Clos fabric; each GPU-affinitized NIC has one port.
SLI	Service Level Indicator; a quantifiable measure (e.g., p99 latency, error rate, throughput). The fabric and observability stack measure SLIs.
SLO	Service Level Objective; target for an SLI (e.g., p99 < 50 ms). Operators define SLOs based on business needs.
SO-C	Scale-out Connectivity, Network providing connectivity for CPU, gateway, and ingress services. Converged with storage in both Minimal and Distributed profiles. Previously referred to as "front-end" in some contexts.
SONiC	Software for Open Networking in the Cloud; an open-source network operating system (Linux Foundation project) designed to run on various switch hardware.
Switch (CRD)	Resource describing a fabric switch (role, profile, ASN, addressing).
Switch Profile	Hardware capability abstraction (ports, breakout, speed/FEC, platform parameters).
Tenant	Isolated workload domain in a multi-tenant cluster, typically mapped to a VPC.
TH5	Broadcom Tomahawk 5; 800G-class switch silicon referenced in this RA.

Term	Definition
VIP	Virtual IP address used for SO-C ingress (load balancer/gateway).
VLAB	Hedgehog Virtual Lab; a virtualized environment that runs the same Hedgehog controller and Broadcom Enterprise SONiC used in production, on virtual SONiC switches. Enables operators to validate configurations, practice lifecycle operations, and build familiarity without physical hardware. See also XOC-Virtual.
VNI	VXLAN Network Identifier; per-segment identifier in overlays.
VPC	Hedgehog overlay abstraction; maps to a VRF in the data plane. Subnets map to VNIs.
VPC API	Hedgehog Kubernetes CRDs (vpc.githedgehog.com) defining logical overlays and external connectivity.
VPCAttachment	Binds a server/port to a VPC (assigns membership and tagging).
VPCPeering	Routed connectivity between VPCs (policy-controlled).
VRF	Virtual Routing and Forwarding instance (tenant routing context).
VXLAN	Data-plane encapsulation for L2/L3 overlays over IP underlay.
Wiring API	Hedgehog Kubernetes CRDs (wiring.githedgehog.com) defining physical topology (Switch, Server, Connection).
XOC-N	xPU Open Cluster for N xPUs; a cluster architecture composed of one or more OPG-M building blocks connected into a scale-out domain. 'N' is total xPU count.

Term	Definition
XOC-Virtual	A composition in the companion repository designed for use with Hedgehog VLAB. Provides virtual servers and a representative network topology for hands-on operator training and configuration validation without requiring physical switch hardware.
xPU	Accelerator processing unit (such as GPU, TPU, IPU) used generically in OCP contexts to refer to any specialized processor designed for acceleration.
xPU-node	Server form factor containing xPUs where the scale-up domain is contained within the server chassis (this RA targets 8 xPUs per node). Independent of cooling modality.
YAML	Yet Another Markup Language format for CRD artifacts.
ZTP	Zero-Touch Provisioning; automated switch bring-up using ONIE and controller-driven workflows.

Appendix A: DSCP and Queue Mapping

This appendix summarizes the traffic classes active when RoCE is enabled for Distributed Inference back-ends.

A.1 Active Traffic Classes

When `spec.roce: true` is set on a back-end switch, three traffic classes (TCs) are active:

Traffic Class	TC	Behavior	Notes
Best Effort	TC0	Default forwarding	All non-marked traffic
RDMA (RoCE)	TC3	Lossless (PFC + ECN)	Scale-out RDMA flows; hosts mark with designated DSCP
Control	TC6	Protected	Control-plane traffic

Minimal Inference omits the back-end network and does not enable RoCE; all traffic uses standard SONiC forwarding (TC0).

A.2 Training vs Inference Defaults

Training (back-end heavy): RDMA class enabled by default; broader use of lossless on back-end.

Inference (Minimal): RDMA class not applicable; standard SONiC forwarding for all traffic.

Inference (Distributed): RDMA class enabled for scale-out fabric only; SO-C forwarding posture is the same as Minimal.

Date: April 2026

A.3 Implementation Notes

Hedgehog renders QoS configuration declaratively via the controller. When RoCE is enabled, PFC, ECN, and DSCP-to-TC mappings are applied automatically -- no manual switch-level configuration is required. Keep mappings consistent across all switches in the fabric by managing configuration through CRDs.

Appendix B: SLI Recommendations

This appendix lists recommended Service Level Indicators (SLIs) and collection methods for inference fabrics. Operators define SLO targets; the fabric exports SLIs with appropriate labels. Where feasible, scope metrics per-tenant.

B.1 Network SLIs (Fabric)

- Tail latency histograms (p95/p99), measured at gateway instances or application endpoints.
- Per-link utilization for capacity monitoring and load-balancing health.
- Pause/PFC events (only for RDMA traffic class when RoCE is enabled on back-end switches).

Labels: tenant/VPC/VRF, device/port, direction, site/cluster.

B.2 Gateway SLIs

- Connections per second (new and total), handshake latency, error rates.
- Throughput (ingress/egress), concurrent connections.
- CPU/memory utilization per instance.

- NAT table utilization; per-tenant connection/session counts (stateful NAT), and gateway error rates.

- Health state and drain events during rolling updates.

B.3 Infrastructure SLIs

- BGP session health (count, flaps, time to converge).

- Route churn rates; EVPN Type-2/Type-5 presence checks.

- Device health: temperature/PSU/fan; platform inventory drift.

B.4 Collection and Export

- Collect device metrics and telemetry via gNMI where supported.

- Use Alloy (Grafana's OpenTelemetry collector) to export to the LGTM stack (Loki, Grafana, Tempo, Mimir).

Appendix C. Composition Variant Summary

This Reference Architecture is accompanied by a companion repository [18] containing composition variants with deployable artifacts for inference-optimized fabrics. The repository is organized by size tier and profile, with each variant providing a complete set of network design assets. The network designs are independent of cooling modality and rack density — the same composition applies to air-cooled and liquid-cooled deployments at any server-per-rack density. Operators should validate optic selections against site-specific cable distances.

Repository Structure

```
inference-ra/compositions/
├── OPG-32/
│   └── minimal-fe-converged/
├── OPG-64/
```

Date: April 2026

```
│   ├── minimal-fe-converged/
│   └── clos-ro--cx7-1x400g--bf3-2x200g--storage-conv-2x200g/
├── OPG-128/
│   ├── minimal-fe-converged/
│   └── clos-ro--cx7-1x400g--bf3-2x200g--storage-conv-2x200g/
├── OPG-256/
│   └── clos-ro--cx7-1x400g--bf3-2x200g--storage-conv-2x200g/
├── XOC-64/
│   └── 1x-OPG-64/
│       ├── minimal-fe-converged/
│       └── clos-ro--cx7-1x400g--bf3-2x200g--storage-conv-2x200g/
├── XOC-128/
│   └── 2x-OPG-64/
│       ├── minimal-fe-converged/
│       └── clos-ro--cx7-1x400g--bf3-2x200g--storage-conv-2x200g/
├── XOC-256/
│   └── 2x-OPG-128/
│       ├── minimal-fe-converged/
│       └── clos-ro--cx7-1x400g--bf3-2x200g--storage-conv-2x200g/
└── XOC-512/
    ├── 2x-OPG-256/
    │   └── clos-ro--cx7-1x400g--bf3-2x200g--storage-conv-2x200g/
    └── 4x-OPG-128/
        └── clos-ro--cx7-1x400g--bf3-2x200g--storage-conv-2x200g/
```

Profiles

Two inference profiles drive variant selection. Both are defined in Section 4.6 of this RA:

- **Minimal Inference (minimal-fe-converged):** Converged SO-C + Storage fabric with no back-end network. Right-sized for single-server model serving where all models fit within a single xPU-node. Gateway is core for north-south ingress/egress. No RDMA; standard SONiC forwarding for all traffic.
- **Distributed Inference (clos-ro):** Adds a scale-out back-end fabric for cross-host parallelism (model sharding, pipeline parallelism, distributed batching, shared caches).

Lossless RDMA transport is enabled by setting `spec.roce: true` on back-end switches. The converged SO-C + Storage front-end fabric is identical to Minimal.

Variant Coverage

The following table summarizes the 14 composition variants provided in the companion repository:

OPG Building Blocks

Size	Profile	Topology	Scale-out NIC	Frontend NIC	Servers	xPUs
OPG-32	Minimal	No back-end; converged FE+Storage	—	BF3 2×200G	4	32
OPG-64	Minimal	No back-end; converged FE+Storage	—	BF3 2×200G	8	64
OPG-64	Distributed	Clos rail-optimized	CX7 1×400G	BF3 2×200G	8	64
OPG-128	Minimal	No back-end; converged FE+Storage	—	BF3 2×200G	16	128
OPG-128	Distributed	Clos rail-optimized	CX7 1×400G	BF3 2×200G	16	128
OPG-256	Distributed	Clos rail-optimized	CX7 1×400G	BF3 2×200G	32	256

XOC Cluster Compositions

Tier	Composition	Profile	Topology	Scale-out NIC	Servers	xPUs
XOC-64	1× OPG-64	Minimal	No back-end; converged FE+Storage	—	8	64
XOC-64	1× OPG-64	Distributed	Clos rail-optimized	CX7 1×400G	8	64
XOC-128	2× OPG-64	Minimal	No back-end; converged FE+Storage	—	16	128
XOC-128	2× OPG-64	Distributed	Clos rail-optimized	CX7 1×400G	16	128
XOC-256	2× OPG-128	Minimal	No back-end; converged FE+Storage	—	32	256
XOC-256	2× OPG-128	Distributed	Clos rail-optimized	CX7 1×400G	32	256
XOC-512	2× OPG-256	Distributed	Clos rail-optimized	CX7 1×400G	64	512
XOC-512	4× OPG-128	Distributed	Clos rail-optimized	CX7 1×400G	64	512

All variants share the following defaults unless otherwise noted: BlueField-3 2×200G frontend NICs with HostBGP multi-homing across two leaves, converged SO-C/storage, and Celestica DS5000 switches.

Profile Availability by Size

Size	Minimal	Distributed
OPG-32	Yes	—
OPG-64	Yes	Yes
OPG-128	Yes	Yes
OPG-256	—	Yes
XOC-64	Yes	Yes
XOC-128	Yes	Yes
XOC-256	Yes	Yes
XOC-512	—	Yes (two composition options)

Minimal Inference is available at smaller tiers where single-server serving is most common. Distributed Inference covers the full range for deployments requiring cross-host parallelism. At OPG-256 and XOC-512 scale, only Distributed variants are provided — at these sizes, the back-end network is expected for cross-host workloads such as model sharding and distributed batching.

Topology Types

- **Minimal (no back-end):** A converged SO-C + Storage fabric serves all non-management traffic. Gateway servers attach to border leaves for ingress/egress. No scale-out back-end fabric; no RDMA. Ideal for inference workloads where each model fits within a single xPU-node and traffic is predominantly north-south.

- **Clos rail-optimized (clos-ro):** Adds a folded leaf-spine back-end fabric with rail-optimized server-to-leaf wiring — all NICs with the same GPU-affinity index (rail) connect to the same leaf or contiguous set of leaves. Same switch counts as a standard Clos.

Enables cross-host RDMA for model sharding, pipeline parallelism, and distributed caching. The converged SO-C + Storage front-end fabric is unchanged from Minimal.

Assets Per Variant

Each variant folder contains the following assets:

- **connectivity-map.csv** — End-to-end device and port mapping for every connection in the composition. Serves as the single source of truth for cabling, labeling, and validation.
- **bom.yaml** — Bill of materials with switch, optic, cable, and gateway quantities and specifications.
- **wiring.yaml** — Hedgehog Kubernetes CRD wiring definitions. This file serves as the complete initial fabric configuration: operators supply these CRDs to the Hedgehog fabricator tool (hhfab) and the controller configures the entire fabric through zero-touch lifecycle management — no manual switch CLI required.
- **vpc.yaml** — Hedgehog VPC CRDs defining overlay networks, tenant VPC attachments, and external connectivity patterns. For Minimal Inference, this covers the converged SO-C + Storage and in-band networks. For Distributed Inference, it additionally defines the scale-out back-end network with RoCE enablement.
- **diagrams/ directory** — Topology diagrams providing visual reference for the composition layout.

Some variants also include a `generated/` directory containing build artifacts: topology authoring inputs, Hedgehog fabricator (hhfab) validation logs, NetBox-importable inventory exports, and generated wiring files.

Variant Naming Convention

Variants use two naming patterns:

Minimal Inference variants:

`minimal-fe-converged`

No back-end network; converged front-end and storage on a single fabric. The name reflects the profile directly.

Distributed Inference variants (tokenized):

`<topology>--<scale-out-NIC>--<frontend-NIC>--<storage>`

Tokens:

- **Topology:** `clos-ro` (rail-optimized Clos)
- **Scale-out NIC:** `cx7-1x400g` (ConnectX-7, 1×400G per xPU)
- **Frontend NIC:** `bf3-2x200g` (BlueField-3, 2×200G per server)
- **Storage:** `storage-conv-2x200g` (converged with SO-C, 2×200G)

Example: `clos-ro--cx7-1x400g--bf3-2x200g--storage-conv-2x200g`

XOC-Virtual (VLAB Composition)

The companion repository includes an XOC-Virtual composition that provides a ready-to-use virtual lab topology. Operators can use this composition to:

- Instantiate a complete Hedgehog-managed inference fabric in the VLAB environment without physical switch hardware
- Practice the full lifecycle: ZTP enrollment, wiring CRD application, VPC tenant onboarding, gateway configuration, and Day-2 operations
- Explore Minimal and Distributed profile differences hands-on

- Validate multi-tenant overlay configuration, gateway NAT/PAT patterns, and operational procedures before production deployment

The XOC-Virtual uses the same CRD format and operational model as all other compositions in the repository. The virtual servers in VLAB provide a comparable network configuration to physical xPU-nodes, enabling a realistic (though not performance-representative) demonstration of the deployment and operational experience.

How to Use

1. Select an inference profile: Minimal (no cross-host communication needed) or Distributed (cross-host parallelism required). See Section 10 for the decision framework.
2. Select a size tier matching your target xPU count (OPG-32 through XOC-512).
3. Review the connectivity map (connectivity-map.csv) to understand port assignments and cabling.
4. Apply Hedgehog Kubernetes CRDs: `kubectl apply -f wiring.yaml && kubectl apply -f vpc.yaml` — or supply them to hhfab for a complete fabric build.
5. For Distributed Inference, enable RoCE on back-end switches by setting `spec.roce: true` in the Switch CRD.
6. Validate using the connectivity map against physical cabling and the BOM against procurement.
7. Adapt optics selections for site-specific distances post site-survey; defaults use flexible-reach DR/SR-class optics for broad compatibility.
8. Use the XOC-Virtual composition with Hedgehog VLAB for hands-on practice before physical deployment.

Relationship to Training RA Companion Repository

The AI Training Fabric RA has its own companion repository [19] with training-optimized compositions. The two repositories share the same asset format (connectivity maps, Hedgehog CRDs, BOMs, diagrams) and the same operational model. Key differences:

- **Training variants** emphasize single-homed Clos vs rail-optimized topology selection, single-plane vs dual-plane configurations, and scale-out backend optimization for collective operations (NCCL/RCCL).
- **Inference variants** emphasize profile selection (Minimal vs Distributed), gateway prominence for north-south serving, and multi-tenant overlay design.
- Both repositories use Hedgehog wiring CRDs that can be applied unchanged to the same controller and switch hardware. An operator deploying both training and inference clusters in the same facility uses the same tools and operational model for both.